



Accelerating Development with dbt Copilot

Jerrie Kumalah Kenney and Treven Mull

October 13-16, 2025

CRAFTED BY



C^x Sandbox Account Setup

- **Log out** of any dbt accounts or use private/incognito session
- **Go to** workshops.us1.dbt.com/workshop
 - Select workshop **Accelerating Development with dbt Copilot**
 - Passcode: **Coalesce2025!**
- **Take a picture** of generated User ID and Password
- Create branch using your name (**example feature/tmull_training**)
- Type **dbt build** in the command bar at the bottom and hit ENTER to build the project for the first time!



Workshop Signup

Please provide your first name, last name, and which workshop you will be attending. After submitting this form, you may log into the new account for your workshop.

First Name

Last Name

Workshop

Passcode

☐ I agree to dbt's [Terms of Service](#)

 Complete Registration



Agenda

1

Introductions

2

**AI in Analytics
Engineering**

3

Effective Prompting

4

**Leveraging External
Assets**

5

Jinja Enabler

6

**Troubleshooting
Partner**



Meet the speakers



Jerrie Kumalah Kenney

Resident Architect

dbt Labs



Meet the speakers



Treven Mull

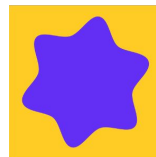
Resident Architect

dbt Labs



Introduction:

AI in Analytics Engineering





Today's Reality: AI as a Co-Pilot

AI is moving away from being a novelty to becoming a necessity, automating the most time-consuming parts of our workflow



Automates repetitive tasks (SQL generation, testing, docs)



Accelerates development & iteration cycles



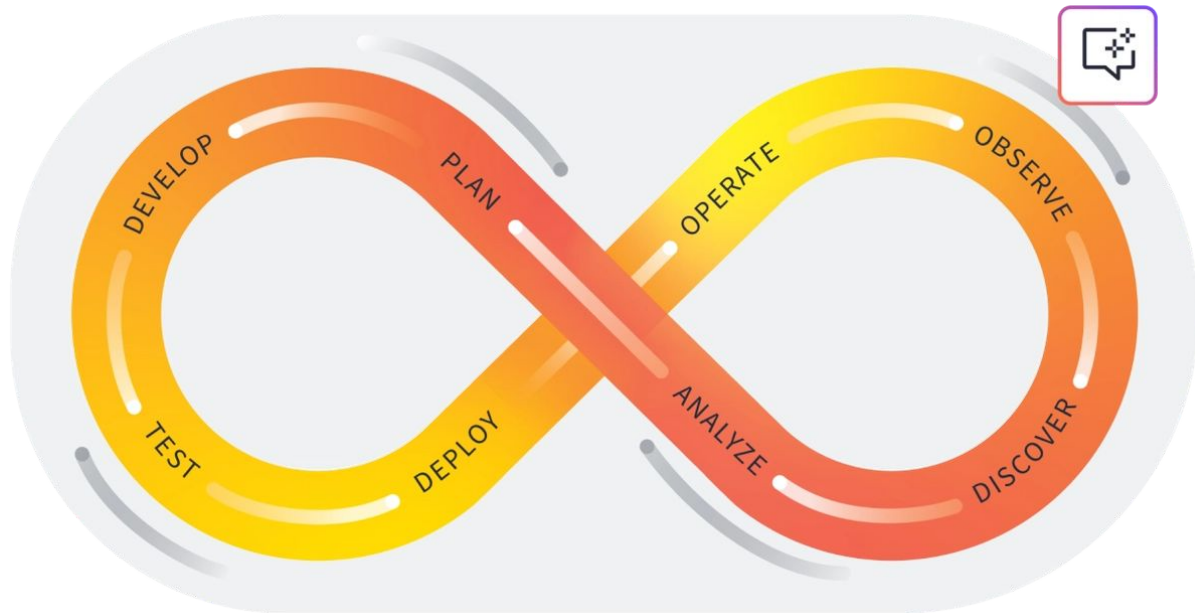
Acts as an intelligent partner, not just a tool



Today, dbt Copilot can be found in three key stages of the ADLC, the develop, test, and analyze stage

..with plans to expand and cover every stage in the future

Accelerate developer productivity with context-aware AI in Studio





Auto-generate SQL using everyday language in Studio

Use everyday language in Studio

Manual SQL generation can be repetitive and prone to errors.

With dbt Copilot:

- Saves time and reduces errors
- Scaled easily with complexity
- Leverages context—from models purpose, lineage and metadata to generate better outputs
- Enables many use cases including
 - Advanced transformations
 - Bulk edits
 - Authoring regex
 - etc.



Auto-generate documentation in Studio using model code

Deliver documentation in minutes

Documentation of data can be a tedious experience.

With dbt Copilot:

- Improve data discovery
- Focus on documenting unique logic
- Leverages metadata to generate better, clearer descriptions for models, columns and metrics



Auto-generate
semantic
models and
metrics in the
Studio

Fast track your semantic layer strategy

Managing YAML to build your semantic layer can be daunting.

With dbt Copilot:

- Reduce time spent writing boilerplate YAML
- Accelerate and scale your semantic layer strategy
- Leverages context from common metrics and dimensions across projects to ensure consistency



Auto-generate data tests in the IDE

Achieve higher test coverage, instantly

Testing is vital for catching production issues but often time-consuming.

With dbt Copilot:

- Automatically generate data tests or scaffold out YAML files (not_null, unique, and relationship test)
- Leverages metadata and context from dbt projects to ensure consistency and accuracy
- Coming soon: Auto-generate unit tests to validate complex logic (regex, date math, etc.)



Accelerating Your Workflow with dbt Copilot



AI-powered assistant integrated throughout dbt

Automates tasks like debugging and SQL generation by using your project's unique metadata for context-aware suggestions.



Auto-generate documentation, tests, and semantic models

Instantly creates docs, tests, and models with a single click to accelerate development and enforce best practices.



Natural language SQL generation

Generates and edits SQL directly in the IDE using plain English, allowing you to focus on modeling logic instead of syntax.

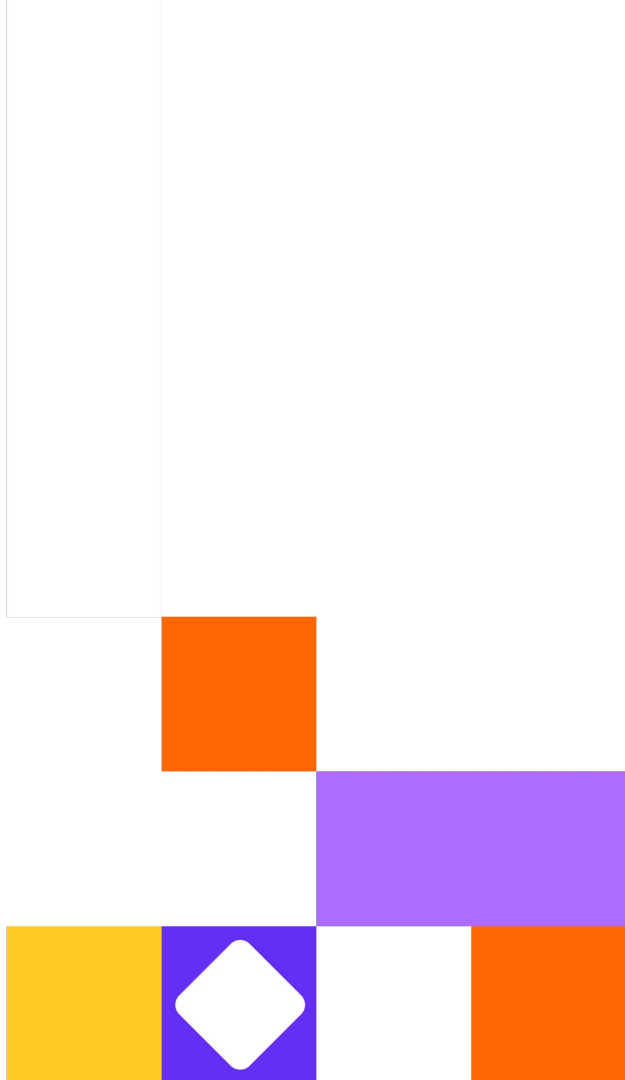


Chat with your data for instant insights

Empowers business users to self-serve by asking questions in natural language and getting immediate answers from the dbt Semantic Layer.



Effective Prompting





Writing **effective** prompts for SQL generation

Provide Rich Context



Include table names, column types, and example values

- how they relate to each other (e.g., **orders** joins to **customers** on **customer_id**)
- **created_at** is a timestamp
- **status** is a string with values like "active" or "pending"



Describe the business question, not just the output

- Avoid purely technical prompts like "write a query using table X"
- Instead of saying "count users," say "**count active users per week to analyze engagement trends.**"
- Describe the decision or insight the query supports (e.g., optimizing marketing spend, identifying churn risk).



Writing **effective** prompts for SQL generation

Be Clear and Explicit



Define the expected output clearly

- Mention the expected columns in the final result
- State whether results should be sorted, limited, or filtered in any special way (e.g., “**top 10 products by revenue**”).
- If applicable, include examples of the output format (e.g., “**I need a column like conversion_rate as a percentage**”).



Break complex logic into smaller, ordered steps

- Write multi-part queries as a sequence of instructions: e.g., “**First filter the dataset... then calculate... then join...**”
- Avoid asking for everything at once; iterative prompting yields better results.



Provide rich context

Coffee Shop Punch Card

You run a neighborhood café. Folks get a free drink after 10 visits.

- **What to give Copilot:**
 - “We’ve got customers, their subscriptions, and their app activity. Customers connect to subscriptions, and activity shows what they do week to week.”
 - The point: You’re giving Copilot a map so it doesn’t wander.
- **What you ask Copilot:**
 - “Help me see weekly regulars and which punch-card folks become subscribers to our ‘Beans of the Month.’”
- **Why it works:**
 - Clear people, clear behavior, clear relationship. Copilot knows what to count and how to connect it.



State the
business
question,
not just the
output

The Sneaker Drop

You run an online sneaker shop. New drop, limited time.

- **What to give Copilot:**
 - “We launched a 14-day trial. Did it lift weekly engagement and upgrades? If yes, we’ll scale it. If not, we’ll tweak the message.”
 - The point: Copilot understands the decision, so it aims at the metric that matters.
- **What you ask Copilot:**
 - “Show me weekly browsers who became buyers, and whether trying the 3D preview led to more purchases.”
- **Why it works:**
 - You’ve described the moment, the behavior, and the decision: keep the 3D preview or not.



Be clear
and explicit
about the
result

The Fitness Challenge

You run a fitness app with a 2-week challenge.

- **What to give Copilot:**
 - “Give me a weekly trend with the date, active folks, and a simple ‘engagement per person.’ Then a summary by launch week with ‘trial starts,’ ‘upgrades in 30 days,’ and an ‘upgrade rate’ as a percentage.”
 - The point: Copilot stops guessing.
- **What you ask Copilot:**
 - “Each week, show active challengers and total workouts. By challenge start week, show how many upgraded to paid within 30 days and what their average workouts looked like.”
- **Why it works:**
 - Specific metrics that ready to present.



In practice

Let's recreate the
customers model!



Exercise #1

05:00

Open [_workshop_resources/exercises/exercise1.md](#)



Leveraging External Assets



Leveraging External Assets

Identifying External Assets for Effective Context

Surface business logic & transformations

- Documentation on metrics definitions, filters, and aggregation rules prevents the model from guessing.

Include relevant sample data

- Small, representative slices of data illustrate edge cases and expected formats without overwhelming token limits.

Capture dependencies and lineage

- Source-to-target mappings and upstream model references guide the model's join logic.



Identifying external assets for effective context

Surface business logic and transformations

Concept:

An “active customer” badge is not a metric definition.
The rule behind it is.

What to give Copilot:

“Active customer means at least one paid purchase in the last 90 days, excluding refunds. Net revenue = gross minus discounts and returns.”

- **External assets:**

- Metrics glossary or KPI catalog in your wiki or BI tool's definitions pane
- Product requirements docs or experiment briefs defining success metrics and filters



Identifying external assets for effective context

Include relevant sample data

Concept:

Swatches, not the whole fabric roll. A few truthful rows reveal edge cases.

What to give Copilot:

orders sample

- order_id: 101, customer_id: C-12, order_date: 2025-07-01, gross: 120.00, discount: 20.00, returned: false
- order_id: 102, customer_id: C-13, order_date: 2025-07-03, gross: 59.99, discount: null, returned: false
- order_id: 103, customer_id: C-12, order_date: 2025/07/05, gross: 80.00, discount: 0.00, returned: true

customers sample

- customer_id: C-12, created_at: 2024-11-10T09:00:00Z, status: active
- customer_id: C-13, created_at: 2025-06-20T15:45:00Z, status: pending

External assets:

- BI exports or CSVs
- API docs with example payloads for upstream sources



Leveraging External Assets

Finding Workflow Stages Where Copilot Adds the Most Value

Model skeleton creation

- Generate initial SELECT statements, joins, and filters from table metadata, accelerating the “blank canvas” stage.

Refactoring or adapting existing logic

- Quickly adjust SQL to new schema changes or different source tables.

Exploratory prototyping

- Use the LLM for quick “what-if” variations before refining into production-ready code.



Finding
workflow
stages
where
Copilot
adds the
most value

Model skeleton creation

Concept:

Frame first, finish later. Start with a clean outline you can refine.

What to give Copilot:

"From **stg_orders** and **dim_customers**, draft a minimal model with **order_id**, **customer_id**, **order_date**, **net_revenue = gross - coalesce(discount, 0)**, and join to **dim_customers** on **customer_id**. Filter to the last 30 days for preview only."

External assets:

- Source-to-target mapping sheets listing join keys and transformations
- Data dictionary with primary keys and foreign keys



Finding
workflow
stages
where
Copilot
adds the
most value

Exploratory prototyping

Concept:

Test three definitions, adopt the one that tells the truest story.

What to give Copilot:

"Generate variants of weekly active logic with trade-offs noted."

External assets:

- Marketing briefs outlining attribution windows and cohort rules
- Customer journey maps highlighting key engagement events



In practice

Let's use a
Source-to-Target
mapping document to
create a model!



Exercise #2

05:00

Open [_workshop_resources/exercises/exercise2.md](#)



Copilot as a Jinja Enabler



What is Jinja?

```
select * from {{ ref('customers') }}
```

COMPILES TO

```
select * from my_database.my_schema.customers
```

```
select {{ cents_to_dollars('amount') }} as amount_usd  
from app_data.payments
```

COMPILES TO

```
select (amount / 100)::numeric(16, 2) as amount_usd  
from app_data.payments
```



Copilot as a Jinja Enabler in dbt

Unlocking macro creation for everyone

For Novices: Lowering the Barrier to Entry

- **Guided syntax generation** – Copilot scaffolds macro code from plain-language prompts, reducing the learning curve for Jinja syntax.
- **Practical examples on demand** – Get inline suggestions for loops, conditionals, filters, and reusable logic patterns.



Lowering the barrier to entry

Guided syntax generation

"I know what I want, I just don't know the Jinja for it."

What to give Copilot:

"I need a macro that uppercases a string and trims whitespace. It should safely handle nulls."

Think of it like:

The karaoke prompter. You know the tune, Copilot gives you the words at the right time so you can sing.



Lowering the barrier to entry

Practical examples on demand

"Show me the pattern for looping over columns and applying a filter."

What to give Copilot:

"I have a list of columns. I want a WHERE clause that checks each for non-null, joined by OR."

Think of it like:

Lego instructions. You bring the pieces, Copilot shows the build steps.



Lowering the barrier to entry

Scenario

A junior analyst keeps copy-pasting CASE statements across models.

Ask:

“Turn this CASE pattern into a macro we can reuse. Include a parameter for the threshold and a note about edge cases.”

Outcome:

The team stops reinventing tiny wheels. The first macro becomes a shared tool, not a one-off.



Copilot as a Jinja Enabler in dbt

Shrinking development time



For Advanced Users: Multiplying Impact

- **Accelerated complex logic design** – Copilot handles repetitive scaffolding so you can focus on the nuanced business rules.
- **Documentation in parallel** – Auto-generate comments, parameter descriptions, and usage examples as you code.



Multiplying impact

Accelerated complex logic design

"I need something more abstract: a macro that builds a grouped aggregation with optional filters."

What to give Copilot:

"Draft a macro with parameters for relation, group_by, metrics, and where. Include defaults and guardrails for empty lists."

Think of it like:

Writer's room sprint. You pitch the premise, Copilot drafts the scene beats.



Multiplying impact

Documentation in parallel

"If it isn't documented, it doesn't exist."

What to give Copilot:

"Add a docstring with parameter descriptions and a usage example. Include a note on performance considerations."

Think of it like:

Margin notes in the first draft. No hunting later for what you meant.



In practice

Let's create some macros!



Exercise #3



05:00

Open [_workshop_resources/exercises/exercise3.md](#)



Troubleshooting SQL



Troubleshooting SQL

Why troubleshoot with Copilot?

Copilot acts as a fast, context-aware reviewer for failing SQL. It reads the error, inspects the shape of your query, and suggests the smallest viable fix.

- Shortens the “hunt” phase. You spend less time interpreting error messages and more time validating a concrete change.
- Turns fixes into patterns. Explanations and snippets can be reused across models and teammates.



Troubleshooting SQL

Where's the value?

Faster diagnosis

- Plain-language translation of errors with likely root causes and the exact line to investigate.
- Pattern spotting across common failure modes: type mismatches, ambiguous columns, unexpected nulls, non-deterministic aggregates.

Safer fixes

- Bias toward small targeted changes.
- Suggests alternative fixes with trade-offs so you can pick the safest for your warehouse and data volume.

Better learning

- Generate a one-paragraph “why it broke” plus a snippet you can paste into docs or PR descriptions.
- Encourages preventative guardrails: a small test, a contract tweak, or a comment explaining edge cases.



Troubleshooting SQL

Decode errors and ambiguity

Error text is a clue, not the crime. Pair the message with the context that produced it.

What to give Copilot:

- “Here’s the failing SQL, the exact warehouse error, and which line it points to. Explain in plain terms and propose the smallest fix.”

Evidence:

- Job run logs or query history export
- dbt build logs with model name and invocation flags



Troubleshooting SQL

Reproduce with a minimal example

Shrink the surface area. If you can break it down, you can fix it fast.

What to give Copilot:

- “Create a reduced query using 5–10 sample rows that still triggers the error. Keep columns only if they influence the failure.”

Evidence:

- Staging table or CSV with representative bad rows
- QA test cases that capture the edge condition
- Saved BI drill-down showing the problematic records



Troubleshooting SQL

Align assumptions with the current schema

Queries encode assumptions. When the schema shifts, assumptions need a tune-up.

What to give Copilot:

- “These columns used to be non-null and consistent. Given today’s schema and constraints, where do assumptions no longer hold? Propose guards like coalesce, accepted values, or adjusted join keys.”

Evidence:

- Data contracts listing required fields and allowed values
- Profiling snapshots showing new nulls or distinct count changes
- BI layer definitions for authoritative column meanings



Troubleshooting SQL

Propose fixes and future guardrails

Fix the bug and install bumpers so it doesn't return.

What to give Copilot:

- "Suggest the minimal code change plus a test and doc note. Include a comment explaining trade-offs and a quick benchmark hint."

Evidence:

- Team testing conventions and approved value lists
- Performance guides for the warehouse (e.g., casting costs, join strategies)



In practice

Let's fix some SQL!



Exercise #4

05:00

Open [_workshop_resources/exercises/exercise4.md](#)



Troubleshooting Macros



Troubleshooting macros

Why troubleshoot with Copilot?

From mysterious Jinja to clear next steps

- Copilot reads your macro and parameters to pinpoint where logic diverges from expectations.
- It explains errors in plain language, proposes the smallest safe fix, and suggests tests to keep it fixed.
- **Takeaway:** Faster root-cause, safer changes, fewer repeat fires.



Troubleshooting macros

Validate parameters and defaults

Most macro bugs start with assumptions about inputs.

What to give Copilot:

- “List required vs optional parameters. Suggest sensible defaults, type checks, and early error messages. Propose guards for empty lists and nulls.”

Evidence:

- Macro usage examples from repo history
- Team conventions for parameter naming and defaults



Troubleshooting macros

Confirm what was actually rendered

Compare intent to the SQL your macro produced.

What to give Copilot:

- “Here’s the macro and the rendered SQL side-by-side. Highlight where the SQL diverges from the intended parameters and propose the minimal macro change to align them.”

Evidence:

- Rendered SQL from target/compiled
- Original prompt or spec describing expected behavior
- Macro docstring or README notes on parameters



Troubleshooting macros

Sanitize identifiers, quoting, and whitespace

Jinja often fails quietly when quoting or spacing is off.

What to give Copilot:

- “Review identifier quoting, string quoting, and whitespace in concatenated SQL. Replace brittle concatenation with safer helpers.”

Evidence:

- Warehouse style guide for quoting rules
- Past PRs fixing quoting or casing issues



Troubleshooting macros

Make logic testable and documented

Fix the bug, then add bumpers so it doesn't return.

What to give Copilot:

- "After the minimal code change, draft a docstring with parameter descriptions, a usage example, and propose two tests: one success case, one edge case."

Evidence:

- Team's macro testing approach or patterns
- Glossary definitions for domain-specific terms used in the macro
- Example models that rely on the macro for real behavior



In practice

Let's fix a macro!



Exercise #5

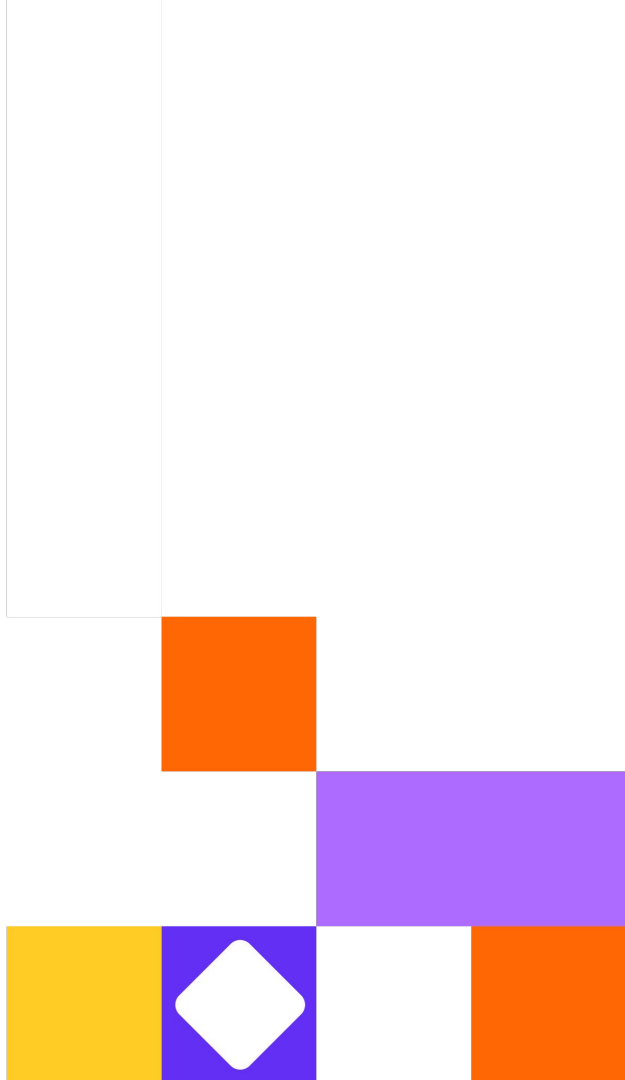
05:00

Open [_workshop_resources/exercises/exercise5.md](#)



Summary:

Key takeaways and discussion





You now have the **framework** to make AI your co-pilot with dbt copilot.

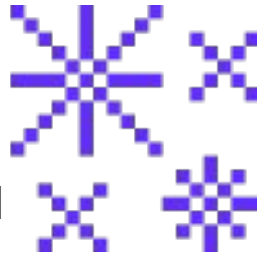
Today, we moved from theory to practice. You've successfully used dbt Copilot to:

- **Master SQL Prompting** to build models with simple, natural language.
- **Amplify Your Existing Workflow** by turning your notes and project context into production-ready code.
- **Generate Jinja Macros** to build more scalable and maintainable systems without the manual effort.
- **Troubleshoot** your code to diagnose issues fast and apply safe, explainable fixes with Copilot.

Your goal now is to embed this into your daily routine. **Start with one task**—automating documentation, generating a test, or refactoring a model—and build the habit from there.



Level Up!



Explore and experiment with these features to go beyond our examples from today

MCP server for AI access to dbt metadata and execution

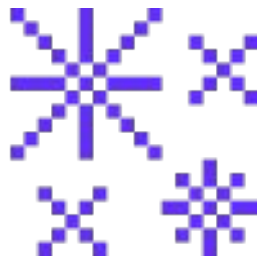
- Expose dbt project structure, lineage, and execution to external AI systems and workflows

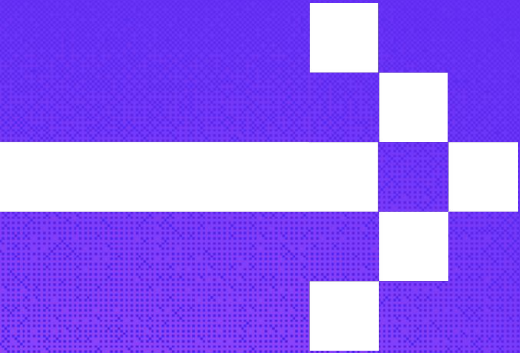
Semantic layer bootstrap with AI

- Accelerate metrics and semantic model authoring by generating YAML and aligning names to your glossary and metadata

Studio and VS Code with Fusion

- Faster, context-aware development experiences leveraging Fusion's SQL comprehension and planned agentic tooling inside editors.





Discussion about other
use cases and tools



Feedback 🙏

